

# CSCI 112

## Programming with C

### LECTURE: 10

J. L. Pach

# OUTLINE:

- Preprocesor
  - Macro substitution
  - File inclusion
  - Conditional compilation
- Breaking down code into files
  - headers
  - source code
- extern & static keyword

# PREPROCESSOR

# The Pre-processor




- The C `Preprocessor` runs before the C program is compiled properly.
- It is a separate program from the compiler programs and, in theory, could be used on any file not just C source files.
- Its main uses are **macro substitution**, **file inclusion** and **conditional compilation**.
- Each source file will normally have a few preprocessor commands, these are, strictly speaking, not part of the C language itself, though they are associated with the language.
- The preprocessor commands each start with a `#` and must either fit on a single source line or must be extended with the use of a - at the end of each line to be continued. eg.: `#include <stdio.h>`

# The C compiler process has four main phases

1. **Preprocessing**: The preprocessor modifies the source code by performing:
    i. **Macro substitution**
    ii. **File inclusion**
    iii. **Conditional compilation**
2. **Compilation**: The compiler translates the preprocessed code into assembly code. This phase includes:
    i. **lexical analysis**,
    ii. **parsing**,
    iii. **code generation**, along with an optional optimization step.
3. **Assembly**: The assembler translates the assembly code into machine code to create an object file.
4. **Linking**: The linker combines all object files and libraries into a single executable program.

# MACRO

# Macro substitution

```
#define symbolic_name replaced_text
```

- A `macro` is a symbolic name that represents a sequence of tokens. It's like creating a shorthand that expands to a longer expression whenever it's encountered in the code.
- Macros allow us to define `constants` whose values cannot be changed during program execution.
- Macros can be used to `conditionally compile` parts of the code, for example, depending on whether a certain definition exists.
- Macros are generally faster than functions but are less safe. Therefore, they should be used with caution.
- **Since macros are substituted before the program runs, they cannot be debugged directly**.

# Macro substitution 1/3

```
#define symbolic_name replaced_text
```

Macros are not terminated with semicolons, unlike regular code statements, making them easily distinguishable. Writing macro-defined constants in uppercase is a best practice to emphasize their immutable nature. While the compiler won't complain, these values cannot be changed during runtime and are not accessible for debugging.

```c
#define PI 3.14159
#define SOME_TEXT "abcde"

#include <stdio.h>
int main()
{
  printf( "%s\n", SOME_TEXT ); // Result: abcde
  printf( "%f\n", PI + 0.5f ); // Result: 3.641590
}
```



```c
/* Replaced to*/

#include <stdio.h>
int main()
{
  printf( "%s\n", /*=====>*/ "abcde" );
  printf( "%f\n", /*=====>*/ 3.14159 + 0.5f );
}
```



# Macro substitution 2/3

Macros are not terminated with semicolons, unlike regular code statements, making them easily distinguishable.

```c
#define square(y) (y * y)
#define merge(left, right) left ## right
#define SOME_TEXT "abcde"

#include <stdio.h>

int main()
{
    char * word1 = "Hello";
    int x = 5;
    printf( "%s\n", SOME_TEXT );      // Result: abcde
    printf( "%d\n", square(x) );      // Result: 25
    printf( "%s\n", merge(word, 1) ); // Result: Hello
}
```

```c
/* Replaced to*/
#include <stdio.h>
int main()
{
    char * word1 = "Hello";
    char * word2 = "world";
    int x = 5;
    printf( "%s\n", /*=====>*/"abcde" );
    printf( "%d\n", /*=====>*/(x * x) );
    printf( "%s\n", /*=====>*/ word1 );
}
```

# MACRO SUBSTITUTION 3/3

```c
#define min(a, b) ( (a) < (b) ? (a) : (b) )
#include <stdio.h>

int main()
{
   printf("%c\n", min('c', 'e'));
   printf("%s\n", min("aac", "aab")); // *
   printf("%d\n", min(3, 2));
   return 0;
}
```

It works because it's a ternary operator!

```c
#include <stdio.h>
int main()
{
    printf("%c\n", ( ('c') < ('e') ? ('c') : ('e') ));
    printf("%s\n", ( ("aac") < ("aab") ? ("aac") : ("aab") ));
    printf("%d\n", ( (3) < (2) ? (3) : (2) ));
   return 0;
}
```

```
Result:
c
aab
2
```

> * but this will not work as expected because it compares the pointers, not the strings themselves

# Conditional compilation

```c
#include <stdio.h>
#define DEBUG 1
#define n 10
int main()
{
    int arr[n];
    int i;
    for (i = 0; i < n; i++)
    {
        arr[i] = rand() % 100;
        #if (DEBUG)   /* #define DEBUG 1*/
            printf("A value of arr[%d] =  %d\n", i, arr[i]);
        #endif
    }
    return 0;
}
```

```c
#if (conditional1)
        statement1;
#elif (conditional2)
        statement2;
#else
        statement3;
#endif
```

```
Result:
A value of arr[0] =  41
A value of arr[1] =  67
A value of arr[2] =  34
A value of arr[3] =  0
A value of arr[4] =  69
A value of arr[5] =  24
A value of arr[6] =  78
A value of arr[7] =  58
A value of arr[8] =  62
A value of arr[9] =  64
```

# Built-in constants in C - examples

`__FILE__` name of the file being compiled

`__DATE__` file compilation date

`__TIME__` file compilation time

```c
#include <stdio.h>

int main()
{
    printf("%s", __TIME__);
    printf("%s", __FILE__);
    printf("%s", __DATE__);
}
```



```
Result:
11:24:35
C:\Users\Jakob\Desktop\CSCI112\main.c
Sep 26 2026
```

# FILE

## inclusion

# FILE INCLUSION

- Using `<>` (angle brackets) – e.g. `#include <file_name.h>`
- Using `""` (double quotes) – e.g. `#include "file_name.h"`

# FILE INCLUSION - <> (ANGLE BRACKETS)

- **Using Standard library search**:
  - When you use angle brackets, the compiler searches for the file in the standard system directories where the compiler expects to find header files for the standard C library.
- **System-wide**:
  - This is commonly used to include standard library (build-in) headers like `<stdio.h>`, `<time.h>`, etc.

# FILE INCLUSION - "" (DOUBLE QUOTES)

- **Local search**:
  - When you use double quotes around a filename in an #include directive, the compiler first searches for the file in the same directory as the current file.
- **Project-specific**:
  - This is useful for including header files that are specific to your current project and are located in the same directory or a subdirectory.

# SUMMARY: HOW THE C PREPROCESSOR WORKS

- The preprocessor works like a *Find and Replace* tool in a text editor — it searches for specific strings (like constant names) and **replaces them with defined values** before the actual compilation begins.
- This means you cannot debug constants or change their values during runtime, because the source code has already been transformed — as if you manually replaced every occurrence of the name with its value.
- **Conditional compilation** allows the program to check whether a constant has been defined (`#ifdef`, `#ifndef`) and **include or skip parts of the code** accordingly. This is useful for writing code that adapts to different configurations, platforms, or versions.

# BREAKING

## down code into files

# SOURCE CODE & HEADER FILES 1/2

To improve code readability, C allows for the separation of `functions`, `data structures`, and `global variables` into separate `files`. This makes the code more clear and easier to maintain.

- **header file**: (with a `.h` extension) contains prototypes of functions, global variables, and structures that are used in multiple source files, providing a way to share information
- **source code file**: (with a `.c` extension) contains the actual implementation of the program's logic, such as function definitions and variable declarations.

To make functions defined in other files visible in a file, it is sufficient to include the header file using #include `name_of_header.h`. There is no need to include files with the `.c` extension.

# SOURCE CODE & HEADER FILES 2/2

- `Function prototypes` serve a dual purpose.
  - They ensure consistency between a function's declaration and its definition. This allows the compiler to verify the compatibility of return types and arguments, preventing hard-to-find errors that often occur when prototypes are omitted.
  - Prototypes are typically placed in separate header files( `.h` ) , especially in larger projects consisting of multiple files. This division enables better code organization.
- The C compiler does not strictly require function prototypes or header files. It's possible to create a new file, write a function, add it to the project, and access it from `main.c` However, in our course, we require the creation of at least one header file containing prototypes for all functions that will be implemented in other files and included in the entire program.

# Example

It's worth noting that header file names and implementation file names don't have to be identical, although this convention is often used to improve project readability

🛠 File: `funs.h`:

```c
int sum(int, int);
int difference(int, int);
```

🛠 File: `funs.c`:

```c
#include "funs.h" /* <== */
int sum(int a, int b)
{
  return a + b;
}
int difference(int a, int b)
{
  return a - b;
}
```

File: `main.c`:

```c
#include <stdio.h>
#include "funs.h"

int main()
{
 int x = 5, y = 3;
 printf("Sum of %d and %d equals %d\n", x, y, sum(x, y));
 return 0;
}
```

Result:

Sum of 5 and 3 equals 8

# REVIEW - PREPROCESSOR DIRECTIVE - #IFNDEF

```c
#include <stdio.h>

#ifndef MAX_SIZE        /* <== */
#define MAX_SIZE 100    /* <== */
#endif                  /* <== */

int main()
{
    printf("MAX_SIZE is %d\n", MAX_SIZE); // Result: MAX_SIZE is 100
    return 0;
}
```



> Checks if a macro is not defined.

# EXAMPLE 1/2

- **Every source file that includes a header will have its contents inserted during compilation.**
- If multiple files include the same header that contains definitions, the linker will report multiple definition **ERRORS** because the same code is compiled more than once.
- To prevent this, header files should use include guards (`#ifndef`, `#define`, `#endif`)

# Example 2/2

## File: funs.h without protection

```
int sum(int, int);              /* ===> */
int difference(int, int);
```



## File: funs.h with protection

```
#ifndef FUNC_H
#define FUNC_H
  int sum(int, int);
  int difference(int, int);
#endif
```



## File: main.c

```
#include <stdio.h>
#include "func.h"

int main()
{
    printf("Sum of %d and %d equals %d\n", x, y, sum(x, y));
    return 0;
}  // Result: Sum of 5 and 3 equals 8
```



## File: func.c

```
#include "funs.h"
int sum(int a, int b)
{
  return a + b;
}
int difference(int a, int b)
{
  return a - b;
}
```



# SUMMARY

*Of all the directives regarding compilation and* `makefiles`*, the* `#ifndef` *directive seems to be the most commonly used, as it prevents the same file from being included more than once in the final output. As can easily be seen, many different files can use the same library.*

# EXTERN KEYWORD 1/2

- In a single code block, we cannot declare two variables with the same name.
- An inner block allows us to declare a variable with the same name inside it, which shadows the variable from the outer block but doesn't destroy it. After the inner block ends, we can access the first variable again.
- Similarly, we cannot declare and initialize the same global variable in two different files.
- If we declare a global variable in one file, to make it visible in another file, we use the extern keyword before the declaration in the second file. In this way, we can access the variable from the other file.

# extern keyword 2/2

```c
#ifndef FUNC_H        /* File: `funs.h` */
#define FUNC_H
  int sum();
  int difference();
#endif
```



```c
#include "funs.h"       /* File: `funs.c` */
extern int x, y; /* <===== */
int sum()
{
  return x + y;
}
int difference()
{
  return x - y;
}
```



```c
#include <stdio.h>        /* File: `main.c` */
#include "funs.h"
int x = 5, y = 3;
int main()
{
 printf("Sum of %d and %d equals %d\n", x, y, sum());
 return 0;
}
```



Result:

Sum of 5 and 3 equals 8

# STATIC

**keyword**

# STATIC KEYWORD 1/3

```c
#ifndef FUNC_H        /* File: `func.h` */
#define FUNC_H
  int sum();
  int difference();
#endif
```



```c
#include "funs.h"        /* File: `func.c` */
static int x = 4, y = 2; /* <===== */
int sum()
{
  return x + y;
}
int difference()
{
  return x - y;
}
```



```c
#include <stdio.h>        /* File: `main.c` */
#include "funs.h"
static int x = 5, y = 3;  /* <===== */

int main()
{
 printf("Sum of %d and %d equals %d\n", x, y, sum());
 return 0;
}
```



Result:

Sum of 5 and 3 equals 6

# STATIC KEYWORD 2/3

- `static` keyword **has two meanings**, depending on where the static variable* is declared:
- Outside a function, `static` variables only visible within that file, not globally
- Inside a function, `static` variables:
  - are still local to that function
  - are initialized only during program initialization*
  - do not get reinitialized with each function call

> * The keyword static can be placed before a function's implementation to restrict its accessibility to the current file. *Static variables are initialized with a default value according to their type, unlike automatic variables.

# static keyword 3/3

```c
#include <stdio.h>
void ticketSale(void);

int main()
{
  ticketSale();
  ticketSale();
  ticketSale();
  return 0;
}
void ticketSale()
{
  static int x = 0; /* initialization at the beginning */
  x++;    /*of the program, not during block execution */
  printf("There are currently %d tickets sold.\n", x);
}
```

```
There are currently 1 tickets sold.
There are currently 1 tickets sold.
There are currently 1 tickets sold.
```

```c
#include <stdio.h>
void ticketSale(void);

int main()
{
  ticketSale();
  ticketSale();
  ticketSale();
  return 0;
}
void ticketSale()
{
  int x = 0;
  x++;
  printf("There are currently %d tickets sold.\n", x);
}
```

```
There are currently 1 tickets sold.
There are currently 2 tickets sold.
There are currently 3 tickets sold.
```

# THANK

You